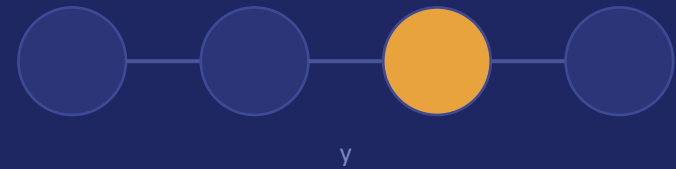


SEQUENCE MODELING

Conditional Random Fields

A discriminative framework for structured prediction over sequences



What we'll cover

01	Motivation	Why structured prediction needs more than per-token classifiers
02	Generative vs. Discriminative	How CRFs differ from HMMs
03	The CRF Model	Graphical structure and the probability formula
04	Feature Functions	The building blocks that encode domain knowledge
05	Worked Example	Full numeric calculation on a POS tagging sentence
06	Training & Inference	Forward-backward, gradient ascent, and Viterbi
07	Applications & Modern Use	NER, chunking, and BiLSTM-CRF hybrids

Conditional Random Fields (CRFs)

A **Conditional Random Field** is a type of probabilistic graphical model used for structured prediction — that is, predicting a whole sequence (or graph) of labels jointly, rather than predicting each label independently.

They're most famous for sequence labeling tasks like part-of-speech tagging, named entity recognition, and information extraction.

The core idea

Given an input sequence $\mathbf{x} = (x_1, x_2, \dots, x_T)$
and a label sequence $\mathbf{y} = (y_1, y_2, \dots, y_T)$,
a CRF models the *conditional* probability $P(\mathbf{y} \mid \mathbf{x})$ directly —
unlike a generative model (like an HMM) which models the joint
 $P(\mathbf{x}, \mathbf{y})$.

Because CRFs are discriminative, they don't need to model how $\mathbf{x}$ is generated, which frees them up to use arbitrary, overlapping features of the input without worrying about independence assumptions between those features.

The math (linear-chain CRF)

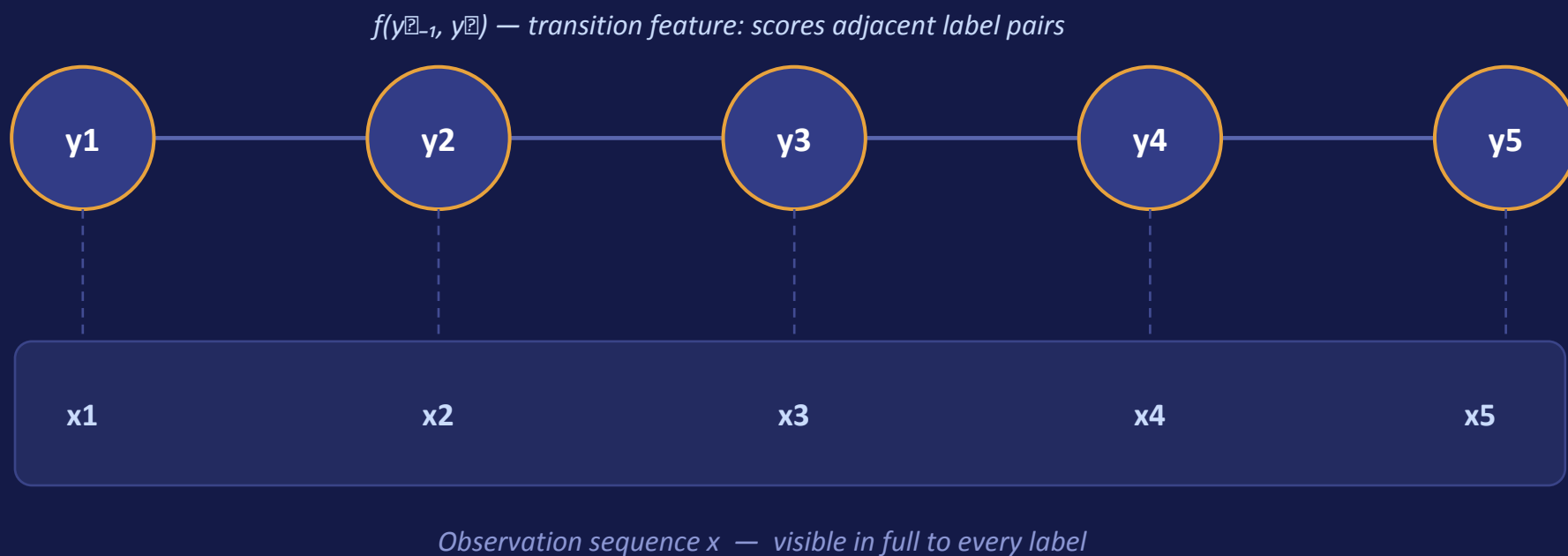
The most common variant is the **linear-chain CRF**, used for sequences. It defines:

$$P(y \mid x) = \frac{1}{Z(x)} \exp \left(\sum_{t=1}^n \sum_k \lambda_k f_k(y_{t-1}, y_t, x, t) \right)$$

Where:

- $\mathbf{f}_k$ are feature functions — e.g., "is the current word capitalized and is the current label B-PERSON?"
- $\boldsymbol{\lambda}_k$ are learned weights for each feature
- $\mathbf{Z}(\mathbf{x})$ is a normalization constant (partition function) summing over *all* possible label sequences, ensuring probabilities sum to 1

Linear-chain CRF structure



Undirected edges

No causal “emits” arrow like an HMM — just a joint potential to be scored.

Global conditioning

Every y_t can use features drawn from anywhere in x , not just x_t .

Whole-sequence scoring

Labels are chosen together, not one greedy token at a time.

The probability formula

$$P(y \mid x) = \frac{1}{Z(x)} \exp \left(\sum_k \sum_t \lambda_k f_k(y_{t-1}, y_t, x, t) \right)$$

 f_k

Feature function — a rule over the label pair, the full input x , and position t

 λ_k

Learned weight for feature k — how much it should influence the score

 $Z(x)$

Partition function — sums $\exp(\cdot)$ over every possible label sequence, so probabilities sum to 1

 $\sum_t$

Sum over every position t in the sequence, from 1 to n

Feature functions encode knowledge

Each $f_i(y_{t-1}, y_t, x, t)$ is a rule that fires (1) or doesn't (0). Weights λ_i , learned from data, decide how much each rule matters.

#	FEATURE FUNCTION F_i	WEIGHT λ_i	WHAT IT CAPTURES
f_1	$y_t = N$ and x_t is capitalized	1.2	Capitalized words look like nouns
f_2	$y_t = V$ and x_t ends in "-s"	1.5	"-s" suggests a 3rd-person verb
f_3	$y_t = N$ and x_t ends in "-s"	0.5	...but could also be a plural noun
f_4	$y_{t-1} = N$ and $y_t = V$	1.5	Noun $\rightarrow$ verb is a common transition
f_5	$t = 1$ and $y_t = N$	1.0	Sentences often open with a noun

WORKED EXAMPLE

Scoring “Time flies”

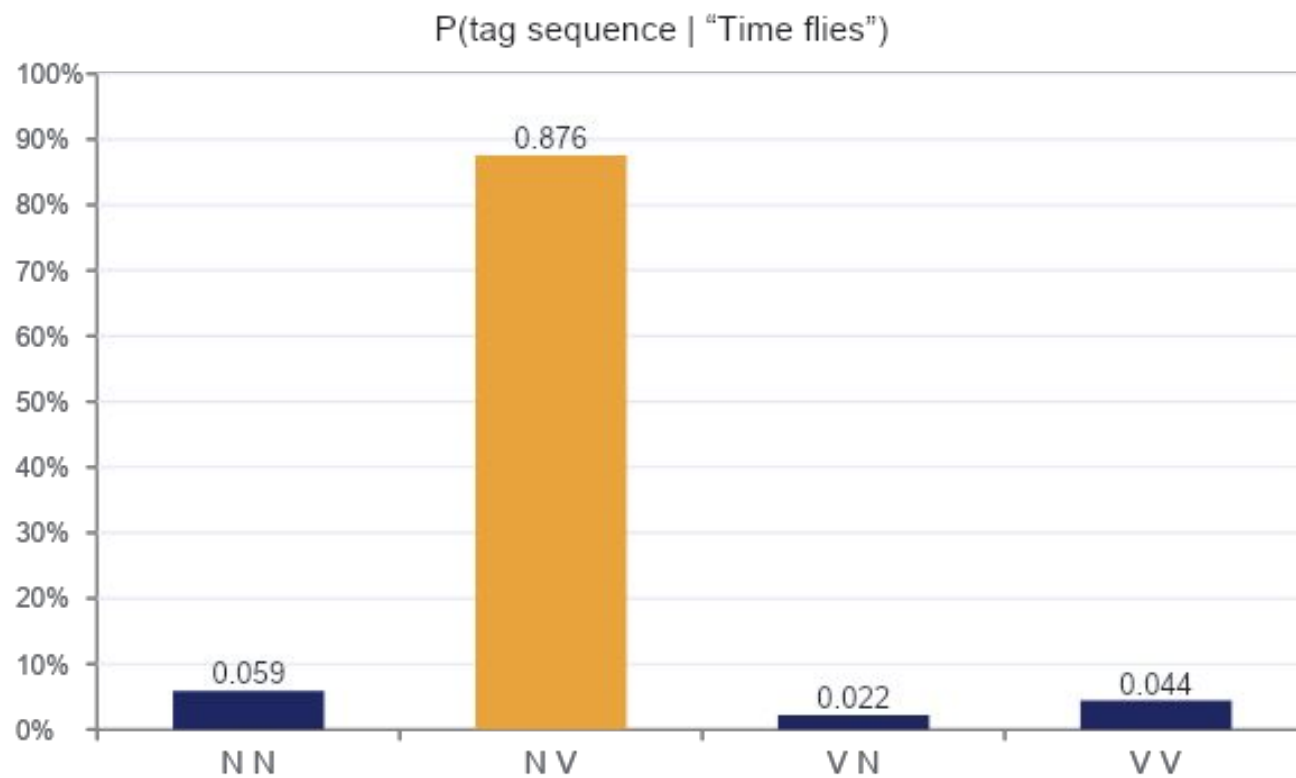
Tags $\in \{N, V\}$. Using the transition + emission weights from earlier, we score all 4 possible tag sequences for $x = (\text{Time, flies})$.

SEQUENCE Y	SUM OF WEIGHTED FEATURES ($\sum \lambda_{ij} F_{ij}$)	EXPONENT	EXP($\cdot$) = SCORE(Y)
N N	1.0 + 2.0 + 0.3 + 0.5	3.8	44.7
N V	1.0 + 2.0 + 1.5 + 2.0	6.5	665.1
V N	0.8 + 0.5 + 1.0 + 0.5	2.8	16.4
V V	0.8 + 0.5 + 0.2 + 2.0	3.5	33.1

$Z(x) = 44.7 + 665.1 + 16.4 + 33.1 \approx 759.4$

Normalizing into probabilities

$P(y \mid x) = \text{score}(y) / Z(x)$ — dividing each score by $Z(x) \approx 759.4$ turns raw scores into a true probability distribution.



87.6%

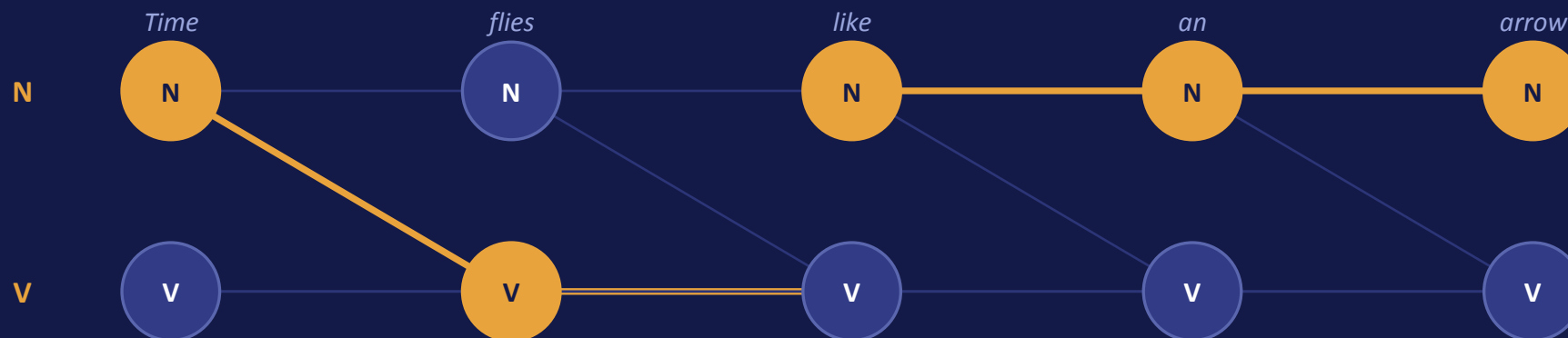
$P(N, V)$ — the clear winner

This matches the natural reading of “Time flies” as noun
→ verb — the transition weight for $N \rightarrow V$ (1.5) reinforced
what the emission features already favored.

*In practice, $Z(x)$ is computed via the forward algorithm — not
brute-force enumeration — since the number of sequences
grows exponentially with length.*

Inference: the Viterbi algorithm

At test time we want the single best label sequence: $y^* = \operatorname{argmax}_y P(y|x)$. Dynamic programming avoids scoring every sequence individually.



Viterbi keeps, at each position, only the highest-scoring path into each tag — backpointers let the full best path be recovered in one backward pass, in $O(n \cdot |\text{tags}|^2)$ time instead of scoring every sequence.

Where CRFs are used

1

Named Entity Recognition

Tagging spans of text as people, organizations, locations, or other entity types

2

Part-of-Speech Tagging

Assigning grammatical categories to every word in a sentence

3

Text Chunking

Grouping tokens into syntactic phrases (noun phrases, verb phrases)

4

Bioinformatics

Gene prediction and protein secondary-structure labeling along a sequence

5

Information Extraction

Pulling structured fields (dates, addresses, prices) out of free text

6

Image Segmentation

Labeling pixels or regions with class labels using spatial CRFs